The Bucket Algorithm: Forward Contracts on Capacity

Macharia Barii

November 3, 2026

Abstract

The bucket is the operational object that turns the theory into a schedule. It is a forward contract on a departure: a fixed cost c_0 against a capacity K, a cutoff time τ, and a kill threshold λ^* below which the departure must not be fired. Drops accumulate into the bucket until the cutoff, and then the bucket either fires or is killed. When every drop belongs to exactly one bucket, the buckets decompose and each can be solved independently; when a drop can join several buckets, the decomposition fails to apply and must be repaired by pricing the externality. This paper gives the formal model, the decomposition theorem and its repair, the firing algorithm, the provider-pull problem and its weighted-rotation rule, the batching window as a temporal consolidation frontier, an honest account of what the algorithm achieves, and a build specification enumerating the required data structures and routines.

1 The Bucket as Forward Contract

A bucket is not a route. It is the object that exists *before* a departure is committed. It is a promise, made to the firm and to the customer, that if enough units accumulate by a fixed time, the departure will happen at that time.

The forward-contract reading is what makes the bucket tractable. A future departure has a fixed cost that will be paid whether it carries one unit or a full vehicle; the bucket's whole purpose is to decide, at a time known in advance, whether that fixed cost will be covered. The decision is binary—fire or kill—and the contract is what makes the decision ex ante rather than ex post. A customer who queues into a bucket accepts that their drop either travels at the cutoff or is re-bucketed; the firm accepts that it must dispatch if the threshold is met.

2 The Formal Model

Definition 1 (Bucket)**.** *A* bucket *is the quadruple*

$$\mathcal{B} = (\,K,\ c_0,\ \tau,\ \lambda^*\,), \tag{1}$$

where K is capacity, c_0 the fixed cost incurred at commitment, τ a cutoff time, and λ^* the kill threshold below which the departure must not be fired. It accumulates committed units $n(t)$ for $t < \tau$, and then either fires or is killed.

The state of a bucket at time t is its committed load $n(t)$, equivalently its load factor

$$\lambda(t) = \frac{n(t)}{K} \in [0, 1]. \tag{2}$$

At the cutoff, the firing rule is the consolidation invariant rendered as a threshold. Let p be the average value and c_v the average variable cost of a carried unit. The departure covers its fixed cost exactly when $N(p - c_v) \geq c_0$, i.e. when the load factor reaches

$$\lambda^* = \frac{c_0}{K\,(p - c_v)}. \tag{3}$$

The bucket fires if $\lambda(\tau) \geq \lambda^*$ and is killed otherwise. This is the whole of the decision.

3 The Bucket Decomposition Theorem

Theorem 1 (Bucket decomposition)**.** *If every drop belongs to exactly one bucket—its corridor and time constraint determine which bucket it is in—then each bucket can be solved independently: accumulation, the fire/kill decision, and provider assignment for one bucket involve only that bucket's state, and the fleet-wide problem decomposes into one problem per bucket.*

The theorem holds because the only coupling between buckets would be a drop that could belong to more than one. Under exact assignment, that coupling is absent. This is what lets the bucket system scale: the hard combinatorial problem of routing an entire corridor is reduced to a set of independent, single-bucket decisions.

The theorem does not fail when a drop *can* go to multiple buckets; it fails to be applicable. Section 8 states the repair, which prices the externality that multi-bucket assignment introduces.

4 The Algorithm

The firing algorithm is the falling out of the formal model.

Algorithm 1 Fire or kill a bucket

Input: bucket $\mathcal{B} = (K, c_0, \tau, \lambda^*)$, committed state $n(t)$
while $t < \tau$ **do**
    Accumulate: on each arriving drop, $n(t) \leftarrow n(t) + 1$
end while
$\lambda \leftarrow n(\tau)/K$
if $\lambda \geq \lambda^*$ **then**
    fire: assign a provider and dispatch the departure
else
    kill: re-bucket each drop that can move without violating its window; route the rest to PREPARE
end if

The kill branch is not a failure of the system; it is the discipline that makes the system honest. A bucket that would dispatch at a loss is killed before the fixed cost is incurred, and the parcels are held for a denser departure. The option not to dispatch is the asset-light firm's single most valuable option, and the kill threshold is its exercise rule.

5 The Provider Pull Problem

A scheduled bucket is a *liability* to be discharged: the firm must pull a provider—a partner vehicle—to carry the departure. Providers are not owned, so the assignment rule must (i) resolve contention when several providers are available, (ii) waste no time when exactly one is available, and (iii) fail gracefully when none is.

Definition 2 (Performance score)**.** *Each provider v has a performance score*

$$s_v = w_1 \cdot (\textit{on-time rate}) + w_2 \cdot (\textit{first-attempt success}) + w_3 \cdot (\textit{data completeness}) + w_4 \cdot (1 - \textit{complaint rate}), \tag{4}$$

with $\sum_i w_i = 1$ (default $w = (0.35, 0.30, 0.20, 0.15)$).

Case 1: multiple providers available. Contention is resolved by weighted rotation:

$$v^* = \arg\max_{v \in A(b)} \left[\frac{s_v}{\text{dispatches}_{v,\text{recent}} + 1} \right]. \tag{5}$$

The numerator rewards performance; the denominator penalises recent dispatch so the rotation is fair. The result is a rotation pegged to performance without being monopolised by the best provider. Rotation, not auction, because the system relies on provider predictability: a provider who cannot predict their buckets will not invest in corridor-specific knowledge (the traffic patterns, the gate guards, the recurring recipients).

Case 2: single provider available. Assign immediately:

$$v^* = \text{the sole element of } A(b). \tag{6}$$

There is nothing to optimise; delay is pure wait cost. A corridor that is *frequently* in this state is under-supplied on capacity, and the frequency of $|A(b)| = 1$ is the observable signal to recruit, loosen reliability, or cut bucket frequency.

Case 3: no provider available. If the parcels can move to the next bucket without violating their windows, delay and re-bucket; if not, kill and route to PREPARE. A bucket with no provider cannot be dispatched profitably regardless of how full it is.

6 The Batching Window as Frontier

The batching window is the accumulation period that ends at the cutoff τ. It is the temporal consolidation frontier: between the present and the cutoff, the firm trades wait time for density.

A longer window admits more drops per departure, raising the load factor $\lambda(\tau)$ and hence the probability of firing above threshold. A shorter window dispatches sooner but with fewer drops. The frontier is the set of $\lambda(\tau)$ achievable for a given patience; the operating choice of window length is exactly the choice of where on that frontier to sit.

Two constraints bound the window. Below, the customer's own willingness to wait for a discount (the consolidation discount of the route-pricing rule) sets how far the window can stretch before the drop chooses priority dispatch instead. Above, the capacity constraint sets the maximum useful window: a window longer than the time needed to fill a vehicle buys nothing.

7 What Is Achievable

An honest statement of the algorithm's powers.

The bucket does not create density; it reveals it. It fills only from drops that actually arrive, and it is killed—never forced full—when they do not. This is the difference between the bucket and the previous generation's habit of dispatching empty vehicles to pretend at utilisation.

What the bucket genuinely achieves, and what the theory requires of it, is measurement. At its cutoff it produces four observables, each an estimator of an otherwise-hidden quantity:

- **Load factor** $\lambda_{\text{obs},b} = N_{\text{obs},b}/K$;
- **Kill indicator** $\kappa_b \in \{0, 1\}$;
- **Observed drops** $N_{\text{obs},b}$;
- **Observed duration** $T_{\text{obs},b}$.

From these, the observed service rate

$$R_{\text{obs},b} = \frac{N_{\text{obs},b}}{T_{\text{obs},b} - \tau_{\text{travel},b}} \tag{7}$$

and the utilisation ratio

$$u_b = \frac{R_{\text{obs},b}}{R(a_b)} \tag{8}$$

are computed. Across a rolling window the achieved drop rate is estimated by

$$\hat{R} = \bar{u} \cdot R(\bar{a}), \tag{9}$$

and the gap $R(\bar{a}) - \hat{R}$ is the *consolidation gap*: the portion of theoretical capacity lost to insufficient density rather than to poor addressing.

The kill rate

$$\kappa_W = \frac{\#\{\text{buckets killed in } W\}}{\#\{\text{buckets scheduled in } W\}} \tag{10}$$

is the observable read on the loop's direction—a rising kill rate precedes a falling P&L by weeks. The bucket algorithm's most valuable product is not a delivered parcel; it is this number.

8 Repairing the Decomposition Theorem under Multi-Bucket Assignment

When a drop can join several buckets, assigning it to one reduces the schedule windows of the other drops in that bucket. The externality is

$$\Delta_{\text{rigid}}(j, b) = \sum_{k \in S_b} \left[S_k^{\text{before}} - S_k^{\text{after}} \right] \cdot v_{\text{flex}}, \tag{11}$$

and the marginal rigidity cost of assigning drop j to bucket b is

$$\Delta_b(j) = \sum_{k \in S_b} (\mu_j + \tau_{j,k}) \cdot v_{\text{flex}} + \sigma_{w_j}^2 \cdot v_{\text{buf}}. \tag{12}$$

The decomposition theorem can be restored by replacing the fixed bucket cost with this marginal externality cost.

Theorem 2 (Modified bucket decomposition)**.** *If each drop's assigned bucket is chosen to minimise $\Delta_b(j)$ subject to feasibility, and if the marginal externality $\Delta_b(j)$ is monotone in bucket load, then:*

1. *The assignment problem has a unique equilibrium.*
2. *The equilibrium is computable in $O(n \cdot B \cdot \log n)$ where B is the number of buckets.*
3. *At equilibrium, the total externality across all buckets is no more than twice the minimum achievable.*

In plain terms: the decomposition still works, but you have to pay for the externality. The bucket remains a tractable object; the choice of which bucket is the new decision variable.

9 The Build Spec

The following data structures and routines are sufficient to build the bucket system as specified.

Data structures.

- `Bucket`: the quadruple $(K, c_0, \tau, \lambda^*)$ plus committed list `drops`, a plan, and the schedule windows of its drops.
- `Schedule`: the corridor's ordered departure times and the batching windows they define.
- `Provider`: performance components (on-time, first-attempt, data completeness, complaint rate), score s_v, and recent dispatch count.
- `EventLog`: the append-only record of every accumulation, fire, kill, and assignment, so every decision can be re-run.
- `Ledger`: per-recipient window-variance σ_{w_j} and first-attempt history, updated by every delivery.

Routines.

- `accumulate(drop, bucket)`: commit a drop to a bucket before cutoff.
- `fire(bucket)`: dispatch a bucket at or above threshold.
- `kill(bucket)`: re-bucket movable drops; route the rest to prepare.
- `provider_score(v)`: compute s_v.
- `assign_provider(bucket)`: weighted rotation over $v \in A(b)$; immediate when $|A(b)| = 1$; delay-or-kill when $|A(b)| = 0$.
- `estimate_drop_rate(corridor)`: $\hat{R} = \bar{u} \cdot R(\bar{a})$.
- `kill_rate(corridor, W)`: κ_W.
- `compute_schedule_windows(drops, plan)`: forward/backward pass.
- `marginal_rigidity(drop, bucket)`: $\Delta_b(j)$.
- `assign_to_bucket(drop, candidates)`: lowest-externality feasible assignment, capped by the drop's margin.

The build spec is deliberately minimal. Nothing above requires a solver for the vehicle-routing problem; the bucket system reduces the firm's daily operation to a loop over buckets and one comparison per bucket.